

twyla

A static-site generator whose only authoring language is typst.

Twyla embeds the typst compiler as a Rust library (the typst crate), not the CLI, so it can: route typst files to URLs, intercept asset resolution to fingerprint/optimize images, walk the compiled document for metadata, emit RSS/Atom, and (eventually) do incremental rebuilds.

The first concrete goal is to port samsartor.com (currently zola + tera + markdown, source at ~/Src/site) to twyla + typst, with HTML AST equivalence as the porting harness.

Why pure typst (no template layer)

Considered: a tera-like template engine layered on top of typst, by analogy with zola layering tera on top of markdown. Rejected — typst already provides the moving parts a template language would re-invent:

- **Inheritance** — `base.html + {% block %}` becomes a typst function that takes a body parameter, with show/set rules supplying the defaults.
- **Filters** — typst functions compose better than pipeline syntax.
- **Shortcodes** — already just typst functions, no separate registry.
- **HTML output** — `html.elem(..)` since typst 0.13 lets us emit arbitrary tag + class + attribute structures, which is what tera shortcodes do today.

Cost of **adding** a template language: two scoping models, two escape-hatch conventions (`| safe` vs typst's `html.frame`), and two contexts for the author to switch between.

Cost of **not** adding one: typst's HTML export is younger than tera, so we may hit ergonomic rough edges. That's where twyla earns its keep — wrapping the crate gives us room to shape the output rather than living with the default.

Open questions

1. **Routing model.** Single typst World, with custom content nodes (working names: `asset`, `resource`, `page`) that auto-split into `html` / `svg` / `pdf` outputs at a derived-or-declared path. Multi-entripoint sugar on top: dropping a new `.typ` into `posts/` creates a post without further config. Open: does the path come from the filesystem (mirror layout), from a `metadata()` call inside the doc, or both? Leaning filesystem-by-default, metadata-override.
2. **HTML AST equivalence.** “Match the zola output up to whitespace” needs a concrete differ — parse both sides (`html5ever` / `scraper`), normalize whitespace + attribute order, compare trees, report structural drift. This is the porting feedback loop; without it, every page becomes manual eyeballing. Must be early infrastructure, not a side tool.
3. **Asset pipeline.** Image references inside typst need to be fingerprinted, optimized, and emitted at stable URLs. Sass and JS bundling — probably shell out to existing tools (current site uses vite) rather than reimplement. Open: does twyla own the dev server, or do we keep vite dev alongside?
4. **Metadata extraction.** Title, description, date, draft, custom `extra.*` fields all live in TOML frontmatter today. In typst they become document metadata + `metadata()` calls, queried after compile. Need to verify what `typst::compile` returns and what's queryable from outside the document.
5. **Compile interface.** One World with many entry points (fast, deduplicates imports, manual source-file lifetime management) vs `typst::compile` per page (simple, slow, no cross-page reuse). Leaning single World.

Milestones

1. **HTML-export prototype.** Pin a typst version. Write three throwaway documents that exercise: heading + paragraph, `<div class="...">` with arbitrary class control, inline raw SVG. Verify

each survives HTML export cleanly. If class control or raw-SVG injection is weak, the design changes — find out now.

2. **AST-diff harness.** Standalone Rust binary that takes two HTML files and reports structural equivalence (whitespace + attribute order normalized). Reused throughout porting; written once.
3. **Port one page end-to-end.** Target: guis-2. Exercises TOML frontmatter, code blocks with syntax highlighting, tables with embedded shortcodes in cells, horizontal rules, and the centered / svg / diagram / image shortcodes. If guis-2 round-trips, most of the site will.
4. **Generalize.** Only after guis-2 matches: routing convention, asset collection, feed generation, dev server.

Findings — experiment 1 (2026-05-17)

Throwaway harness at `src/main.rs` + `experiment.typ`, 115 lines of Rust total. Implements the minimal World, compiles with `typst 0.14.2` + `html` feature, dumps document info, metadata, asset requests, and rendered HTML.

Confirmed:

- **Class control is exact.** `#html.div(class: "container --large")` emits `<div class="container --large">` verbatim. CSS-class-driven porting from zola is unblocked.
- **Document metadata round-trips.** `#set document(title:, author:)` lands in `HtmlDocument::info.title / .author` as expected.
- **metadata(..) round-trips with full structured payloads.** Dicts, datetimes, arrays all survive. Query selector: `Selector::Elem(MetadataElem::ELEM, None)`; unpack via `Content::to_packed::<MetadataElem>()` to read the value field. Label-based lookup also works. **Note:** `Selector::can::<MetadataElem>()` returns 0 hits — wrong matcher despite compiling; use `Selector::Elem` instead.
- **HTML escaping is automatic.** A literal `<` inside `#raw(..)` came out as `<code><</code>`.
- **World::file is the asset hook.** `image("teaser.svg")` triggered exactly one call with `path = "teaser.svg"`. The `FileId` carries `vpath` (the rootless requested path) — enough to drive a content-addressed asset pipeline.
- **Library construction gotcha.** HTML export is gated on `Library::builder().with_features([Feature::Html])`. Not a Cargo feature. Without it, compile errors with “html export is only available when `--features html` is passed” (misleading — there is no such Cargo flag). Always emits a warning even when enabled; that’s fine.

Surprise — and a design fork: `image("teaser.svg")` in HTML mode does **not** emit ``. Typst reads the bytes via our World, then base64-encodes them into a `` data URL inline in the HTML. Twyla cannot just intercept `image()` and rewrite the URL — typst owns the rendering. Three viable paths:

1. Define a twyla-provided `asset("foo.svg")` function in the library that **does not** call `image()` — instead, it returns content built from `html.img(src: <our-derived-url>)`. We do asset interning at the function-call site, emit external `<img>` tags, and never use typst’s `image()` for HTML output. **Probably the right answer** — gives us explicit control of class names, `srcset`, lazy-loading, captions, etc.
2. Post-process the HTML: walk the DOM, extract every data: URL, hash the bytes, write them to disk, rewrite the `src` to the new URL. Works but feels brittle — any future typst optimization to elide tiny images or fold multiple references would silently change our output.
3. Let typst inline assets and accept the data-URL bloat. Fine for tiny decorative SVGs, terrible for the site’s 50KB SVG diagrams. Not serious for our content.

Lean: option 1. The asset/resource/page custom-content-type idea from the earlier sketch maps cleanly here — `asset()` is one of those types, and it bypasses `image()` entirely.

Other observations:

- Typst emits a full HTML5 document by default (doctype, `<html>`, `<head>` with meta charset, viewport meta, `<title>`, author meta). For matching the zola output we'll need to either suppress this and emit our own `<head>` shell, or replace the auto-emitted `<head>` contents via... some mechanism not yet discovered.
- Output is pretty-printed with 2-space indentation. The AST-diff harness can ignore this trivially but worth knowing.
- typst-kit's `Fonts::searcher().include_system_fonts(true).search()` is one line and gave us system fonts with zero ceremony — keep using it during development.

Notes

- VCS: both `~/Src/twyla` and `~/Src/site` are managed with `jj`. **Rule:** always `jj new` **before** editing — `jj` has no staging area, so edits land in whatever revision the working copy points at. Recovering from an accidental commit requires `jj split`, which is interactive and cannot be driven from a non-interactive shell.
- Typst version: TBD (pin when starting milestone 1).
- Tera shortcodes in scope for porting: `centered`, `diagram`, `image`, `math`, `svg`, `tagline`, `var` (`~/Src/site/templates/shortcodes/`).
- Tera templates in scope: `base.html`, `page.html`, `section.html`, `index.html`, `components.html`, `404.html`, `atom.xml`, `paperpillls.html`.
- Known porting hazards: ROT13-encoded email obfuscation in footer, conditional asset loading (`page.extra.tilings`), SVG inlining with font-family rewriting.